



A Comparative Study of Rapidly-exploring Random Tree Algorithms Applied to Ship Trajectory Planning and Behavior Generation

Trym Tengesdal¹ · Tom Arne Pedersen² · Tor Arne Johansen¹

Received: 22 April 2024 / Accepted: 30 December 2024 / Published online: 11 January 2025
© The Author(s) 2025

Abstract

Rapidly Exploring Random Tree (RRT) algorithms, notably used for nonholonomic vehicle navigation in complex environments, are often not thoroughly evaluated for their specific challenges. This paper presents a first such comparison study of the variants Potential-Quick RRT* (PQ-RRT*), Informed RRT* (IRRT*), RRT*, and RRT, in maritime single-query non-holonomic motion planning. Additionally, the practicalities of using these algorithms in maritime environments are discussed and outlined. We also contend that these algorithms are beneficial not only for trajectory planning in Collision Avoidance Systems (CAS) but also for CAS verification when used as vessel behavior generators. Optimal RRT variants tend to produce more distance-optimal paths but require more computational time due to complex tree wiring and nearest neighbor searches. Our findings, supported by Welch's t -test at a significance level of $\alpha = 0.05$, indicate that PQ-RRT* slightly outperform IRRT* and RRT* in achieving shorter trajectory length but at the expense of higher tuning complexity and longer run-times. Based on the results, we argue that these RRT algorithms are better suited for smaller-scale problems or environments with low obstacle congestion ratio. This is attributed to the curse of dimensionality, and trade-off with available memory and computational resources.

Keywords Rapidly-exploring random trees · Comparison study · Ship trajectory planning · Scenario generation · Electronic navigational charts · R-trees · Constrained delaunay triangulation

1 Introduction

1.1 Background

Trajectory planning is an important aspect of ship autonomy, not only in Collision Avoidance Systems (CAS) for safe and efficient voyage [1–3], but also for the safety assurance of the former. To verify CAS safety and compliance with

the International Regulations for Preventing Collision at Sea (COLREG) [4], it will be necessary to conduct simulation-based testing in a diverse set of scenarios [5]. The scenarios must cover varying difficulties with respect to grounding hazards or static obstacles, ships with uncertain kinematics and intentions, and environmental disturbances. Here, it will be important to develop methods for generating interesting and hazardous obstacle vessel behavior scenarios with sufficient variety for the CAS testing. Up until now, this has proved to be a challenging problem not yet solved.

For planning trajectories, roadmap methods such as graph-based search algorithms A* [6] provide guarantees of finding the optimal solution, if it exists, given its considered grid resolution. However, they do not scale effectively with the state space dimensionality and the problem size, further being unable to consider kinodynamic constraints unless e.g. hybrid A*-variants are used. This often makes it necessary to post-process the solution through smoothing or using optimization-based methods to generate feasible and optimal trajectories. Sampling-based motion planning algorithms

✉ Trym Tengesdal
trym.tengesdal@ntnu.no

Tom Arne Pedersen
tom.arne.pedersen@dnv.com

Tor Arne Johansen
tor.arne.johansen@ntnu.no

¹ Department of Engineering Cybernetics, Autoship Centre for Research-based Innovation (SFI), Norwegian University of Science and Technology (NTNU), O. S. Bragstads Plass 2D, Trondheim 7034, Norway

² Det Norske Veritas (DNV), Veritasveien 1, Høvik 1363, Norway

such as RRT here provide a good foundation for tackling complex environments with non-convex grounding hazards, without requiring a discretization of the state space, with RRT*-variants [7] being *asymptotically optimal*, guaranteeing a probability of finding the optimal solution converging towards unity, if it exists, as the number of iterations approaches infinity. The regular RRT-variants are suboptimal due to the Voronoi bias toward future expansion as opposed to iterative graph rewiring based on node costs in RRT* [8]. However, an underlooked aspect of RRTs is that their inherent randomness makes them promising for trajectory scenario generation, being flexible with respect to biased state space exploration. Different variants can be employed for generating different ship behavior scenarios which can cover their maneuvering space.

1.2 Previous Work

Collision-free trajectory planning is a well-studied topic, and we refer the reader to review studies as in [9, 10] for extensive summaries on the various methods proposed previously, and to [11] for an extensive review on RRT planning dated 2016. This brief review focuses on core versions of RRT having been proposed previously, in addition to ship scenario generation and falsification, respectively. The RRT algorithms are *single-query*, and thus they are tailored for planning trajectories or paths from a start position to a specified goal. For information on multi-query planning, the reader is referred to e.g. Probabilistic Roadmap methods (PRM) [12].

1.2.1 Rapidly-exploring Random Trees

The first baseline RRT planner was introduced by [13], with the core concept of incrementally sampling configurations or nodes in the obstacle-free space, and wiring of the tree towards these configurations if the trajectory segments in between are collision-free. However, baseline RRT is only *probabilistically complete* in the sense that the probability of the planner finding a solution approaches 1 as the number of iterations approaches infinity. This issue was addressed in [7] with RRT*, giving *asymptotically optimality* guarantees by introducing optimal selection of node parents during tree wiring through nearest neighbor search, and also by rewiring the tree after a new node has been inserted. However, the RRT-based planning still suffered from slow convergence. Since then, a multitude of approaches has been proposed to remedy this [14], e.g. Smart-RRT* in [15] and IRRT* [16]. The former variant optimizes the found solutions by connecting visible collision-free nodes and utilizing them in the optimized solutions as beacons from which biased samples are drawn at a given percentage. The intelligent sampling procedure, however, only yields improved local convergence

to solutions in the vicinity of the current best solution. IRRT* employs a hyperellipsoid sampling heuristic from a subset of the planning space, formed after an initial solution is found, and which reduces in volume as the planner improves the current best solution. This variant provides formal linear convergence guarantees, although only given the assumption of no obstacles.

More recently, Potential Quick RRT* (PQ-RRT*) has been proposed [17], as a combination of the Artificial Potential Field (APF) based RRT* in [18], and Quick-RRT* in [19]. As RRT* is inherently biased towards the exploration of the obstacle-free configuration space, the APF method adds exploitation features through a goal-biased sample adjustment procedure, while the Quick-RRT* gives an improved convergence rate through ancestor consideration in the wiring and re-wiring. As mentioned, there exist a significant number of other variants in the literature, that e.g. add bi-directional tree growth [20], utilize learning-based steering functionality [21] or combine A* with RRT* in a learning-based fashion [22]. The trade-off between exploitation and exploration in RRT was addressed without nonholonomic system consideration in [23]. A large literature review on sampling methods utilized in RRTs was given in [24], which sheds light on the multitude of variants that have been proposed over the years. In the present work, we only focus on the core versions of directional RRT that have gained traction in the field, namely RRT, RRT*, IRRT*, and PQ-RRT*.

1.2.2 Scenario Generation

This paper also demonstrates a proof-of-concept usage of RRTs for maritime ship behavior scenario generation. We note that this is not new in other domains, as scenario generation and falsification of safety-critical systems have been proposed in e.g. [25] for testing nonlinear systems subject to disturbances, in [26] for generating initial car vehicle states that yield boundary cases where the automated vehicle can no longer avoid collision, and in [27] for adaptive cruise control falsification to search for hazardous leading vehicle behaviors leading to rear-end collisions.

What has typically been done in previous work within the maritime domain is to consider constant behaviors for vessels involved in a scenario [28–31]. The authors of [30] used a Gaussian Process to estimate how CAS scores concerning safety and the COLREG, which guides the selection of scenarios to test the system at hand, based on its confidence level of having covered the parameter space describing the set of scenarios. Zhu et al. [32] proposed an Automatic Identification System (AIS) based scenario generation method. Here, AIS data was analyzed and used to estimate Probability Density Functions (PDFs) describing the parameters of an

encounter, such as distances between vessels, their speeds, and bearings. The PDFs were then used to generate a large number of scenarios for testing CAS algorithms. The goal was to increase the test coverage for such systems, over that which is possible with only expert-designed and real AIS data scenarios. Again, generated vessels all follow constant velocity, which does not always reflect true vessel behavior in hazardous encounters. Furthermore, one can not expect all vessels in a given situation to broadcast information using AIS, thus many times making AIS-generated scenarios partially incomplete.

The authors in [33] used the Deep Q Network (DQN) for building a scenario test suite, based on using a neural network to score the performance of randomly generated scenarios. The performance is calculated based on geometric two-ship COLREG compliance and the risk of collision, where the score is used to determine if a given scenario is eligible for simulation and test suite inclusion. The approach should, however, be refined to account for more navigational factors such as grounding hazards in the performance evaluation. Recently, Bolbot et al. [31] introduced a method for finding a reduced set of relevant traffic scenarios with land and disturbance consideration, through Sobol sequence sampling, filtering of scenarios based on risk metrics, and subsequent similarity clustering. Again, constant behavior is assumed for the vessels, but the process of identifying hazardous scenarios shows promise.

1.3 Contributions

The paper is the first comparison of the previously proposed RRT, RRT*, IRRT*, and Potential Quick-RRT* (PQ-RRT*) together, applied in a complex maritime environment for single-query directional planning with consideration of non-holonomic ship motion. This is contrary to the disregard for nonholonomic vehicle motion and often simpler and regular environments that have typically been used for testing and comparing RRTs in a lot of previous work.

The planners are compared in Monte Carlo simulations for cases of varying complexity, with consideration of nonholonomic ship dynamics. The trajectory length results produced by PQ-RRT* compared to the other RRT planners are analyzed for statistical significance using Welch's Student t -test. The chosen variants RRT, RRT*, IRRT* and PQ-RRT* are considered as they represent core improvements of the RRT algorithm over the last 25 years, which we deemed the most interesting to compare. Comparisons of fusions of RRT* with path complete methods such as A* as in e.g. Wang et al. [22], bi-directional RRTs, and RRT variants with alternative and more sophisticated steering mechanisms [21] are outside the scope of this work. The same goes for the multitude of

sampling strategies proposed in the literature [24]. In addition to the comparison study, the paper also provides guidelines and a discussion around practicalities when applying such algorithms for ship trajectory planning, which can be of use to researchers and practitioners in the field.

As a side contribution, we argue through proof-of-concept cases that RRT-based planners are beneficial for vessel test scenario generation due to their rapid generation of initially feasible, although not necessarily optimal, trajectories. As we do not necessarily require that obstacle vessels follow optimal trajectories, they provide a viable approach for the fast generation of random vessel scenarios used in CAS benchmarking. RRTs can also be used to generate more realistic ship intention scenarios that can be exploited in intention-aware CAS such as the Probabilistic Scenario-based Model Predictive Control (PSB-MPC) [1, 34]. Lastly, the RRTs can be used in frameworks as in Bolbot et al. [31] for finding relevant scenarios, where the RRT sampling heuristics can be tailored to the considered navigational factors.

1.4 Outline

The article is structured as follows. Preliminaries on ship dynamics and control are given in Section 2, and background on trajectory planning and RRT variants in Section 3. Notes on practical aspects to consider when applying RRTs for planning are given in Section 4, whereas results from applying RRTs for ship behavior generation and trajectory planning are given in Section 5. Lastly, conclusions are summarized in Section 6.

2 Preliminaries

2.1 Notation

Vectors and matrices are written as boldface symbols. The Euclidean norm is written as $\|\mathbf{x}\|$ for a vector $\mathbf{x} \in \mathbb{R}^n$, with n denoting the vector space dimension. Similarly, a matrix is written as $\mathbf{M} \in \mathbb{R}^{n \times m}$ and has n rows and m columns.

2.2 Ship Modelling

When planning motion trajectories over longer time horizons, there is seldom a need to consider high-fidelity vehicle models, as modeling errors and disturbances will compound substantially. However, the model should as a minimum take the vehicle's kinodynamical constraints into account. Thus, without loss of generality, we here employ a kinematic ship model on the form $\dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}(t), \mathbf{u}(t))$ with $\mathbf{x} = [x, y, \chi, U]^T \in \mathbb{R}^{n_x}$ consisting of the planar ship

position, course, and speed, with input $\mathbf{u} = [\chi_d, U_d]^T \in \mathbb{R}^{n_u}$ consisting of course and speed autopilot references, and thus $n_x = 4, n_u = 2$. The equations of motion are given by

$$\begin{aligned}\dot{x} &= U \cos(\chi) \\ \dot{y} &= U \sin(\chi) \\ \dot{\chi} &= \frac{1}{T_\chi}(\chi_d - \chi) \\ \dot{U} &= \frac{1}{T_U}(U_d - U)\end{aligned}\quad (1)$$

with time constants $T_\chi > 0$ and $T_U > 0$ depending on the ship type. The ship maneuverability constraints $U_{min} \leq U \leq U_{max}$ and $|\dot{\chi}| \leq r_{max}$ on the minimum speed U_{min} , maximum speed U_{max} and maximum turn rate r_{max} of the ship are considered to ensure kinodynamic feasibility.

RRTs are capable of considering arbitrarily complex ship dynamics and disturbance models, but we again note that the chosen model yields reduced computational requirements and is suitable for planning trajectories covering larger distances. Over large timespans, external disturbances, and modeling errors will compound substantially over the planning horizon, yielding low benefit from using high-fidelity vessel models. Note that for lower-level motion control systems used for tracking the RRT trajectory output, it will be important to more accurately take the ship dynamics into account. Also, when using RRT to generate random obstacle ship scenarios, one seldom has access to the detailed ship model and motion control system employed by the target ship, further making this model suitable.

In summary, the present work utilizes a nonholonomic ship model with time constant parameters that captures the kinematic ship motion. The model parameters must be identified using real ship motion data for it to be usable in practice, to prevent feasibility issues in tracking the planned high-level RRT trajectory. Uncertainties affecting the ship motion due to external disturbances and further model inaccuracies are assumed compensated for by the ship low-level autopilot.

2.3 Line-of-Sight Guidance

To enable lightweight steering functionality in the RRTs and allowing for orientation or course control, we employ Line-of-Sight (LOS) guidance [35] for steering the ship from a waypoint segment from $\{\mathbf{p}_1, \mathbf{p}_2\}$, where $\mathbf{p}_1 = [x_1^{wp}, y_1^{wp}]^T \in \mathbb{R}^2$ is the planar waypoint position. To steer the ship along the straight waypoint segment and towards $\mathbf{p}_2$, the LOS method first finds the path tangential angle

$$\theta_p = \text{atan2}(y_2^{wp} - y_1^{wp}, x_2^{wp} - x_1^{wp}) \quad (2)$$

where $\text{atan2} : \mathbb{R} \rightarrow (-\pi, \pi]$ is the four-quadrant arctangent function. Then, the path deviation $\boldsymbol{\epsilon}(t)$, referenced to the path-fixed frame, is computed as

$$\boldsymbol{\epsilon} = \mathbf{R}(\theta_p)^T (\mathbf{p}(t) - \mathbf{p}_1) \quad (3)$$

with the rotation matrix $\mathbf{R}(\theta_p)$ given by

$$\mathbf{R}(\theta_p) = \begin{bmatrix} \cos(\theta_p) & -\sin(\theta_p) \\ \sin(\theta_p) & \cos(\theta_p) \end{bmatrix} \quad (4)$$

The path deviation $\boldsymbol{\epsilon}(t) = [s(t), e(t)]^T$ consists of the along-track error $s(t)$ and cross-track error $e(t)$, respectively, where the latter is used with the path tangential angle θ_p to set the desired course-over-ground (COG) $\chi_d(t)$ as

$$\chi_d(t) = \theta_p + \arctan\left(-\frac{e(t)}{\Delta}\right) \quad (5)$$

where Δ is the look-ahead distance that determines how fast the ship will turn towards the straight line segment. The desired speed-over-ground (SOG) $U_d(t)$ can, in general, be varying, but is typically set to a constant in the case of nominal trajectory planning and generation. Without loss of generality, we do not consider environmental disturbances. However, note that compensations for slowly varying disturbances can be considered by adding an integral term in Eq. 5. See [35] for illustrations and more information.

3 Trajectory Planning Using Rapidly-exploring Random Trees

3.1 Problem Definition

This section formally defines the considered motion planning problem and the RRT-based algorithms compared in this work: Standard RRT [13], RRT* [7], IRRT* [16] and Potential Quick RRT* (PQ-RRT*) [17]. As mentioned in the introduction, bi-directional variants such as RRT*-connect [20] and combinations of RRT with e.g. path-complete methods such as A* are outside the scope of this work. The following text does not provide an in-depth introduction to each algorithm, and the reader is thus referred to the cited references for more details.

We consider the own-ship motion model $\dot{\mathbf{x}} = f(\mathbf{x}(t), \mathbf{u}(t))$ as given by Eq. 1 with state $\mathbf{x}(t) \in \mathcal{X}$ and input $\mathbf{u}(t) \in \mathcal{U}$, belonging to the configuration space $\mathcal{X} \subset \mathbb{R}^{n_x}$ and input space $\mathcal{U} \subset \mathbb{R}^{n_u}$, respectively. Let $\mathcal{X}_{obst}$ denote the forbidden static obstacle space and $\mathcal{X}_{free} = \mathcal{X} \setminus \mathcal{X}_{obst}$ the static obstacle-free configuration space.

The optimal trajectory planning problem is stated as follows. Let $\mathbf{z}_{start} \in \mathcal{X}_{free}$ and $\mathbf{z}_{goal} \in \mathcal{X}_{free}$ be the initial and final own-ship states, respectively. Further let $\sigma_d : [0, T_{plan}] \rightarrow \mathcal{X}$ be a non-trivial trajectory from start to goal. Then, the objective of the RRT-based planning algorithms is to find the optimal and feasible desired trajectory σ_d^* defined through

$$\begin{aligned} \sigma_d^* &= \arg \min_{\sigma_d} \left(c(\sigma_d) \mid \sigma_d(0) = \mathbf{z}_{start}, \sigma_d(T_{plan}) \right. \\ &\quad \left. = \mathbf{z}_{goal}, \forall t \in \{0, T_{plan}\}, \sigma_d(t) \in \mathcal{X}_{free} \right) \end{aligned} \quad (6)$$

where T_{plan} is the trajectory duration and $c(\cdot) : \mathcal{X} \rightarrow \mathbb{R}$ the planning problem cost function. In summary, the required inputs and resulting output of the RRT planners are

Inputs $\mathbf{z}_{start}$, $\mathbf{z}_{goal}$ and Electronic Navigational Chart (ENC): Starting state, goal state, and ENC containing grounding hazard information, respectively.

Output Collision-free trajectory plan σ_d starting in $\mathbf{z}_{start}$ and ending in $\mathbf{z}_{goal}$.

The output trajectory from the RRT-based planners can be generated offline or online depending on the application and is typically provided as a reference trajectory for the lower-level motion control system onboard the ship to track.

3.2 Core RRT-functionality

To solve the described trajectory planning problem, RRT-based planners incrementally and randomly grow a tree $\mathcal{T} = (\mathcal{V}, \mathcal{E})$ consisting of a state vertex or node set $\mathcal{V} \subset \mathcal{X}_{free}$ that are connected through the directed edges in $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$. Asymptotic optimality in the number of iterations for the planned trajectory is guaranteed in RRT* and its variants through the parent-cost dependent wiring and rewiring of the tree in order to find new minimal cost parents [36]. The standard RRT method does, however, not have this property. We note that the convergence rate is highly dependent on the node sampling procedures, in addition to the tree re-wiring mechanism. Common functions used in all the RRT-based planners are given below.

1) **Sample(i)**: Samples a random state $\mathbf{z}_{rand} \in \mathcal{X}_{free}$. See Section 4.1 for implementation aspects. When applied to random scenario generation, the sampling can be biased towards scenario-related metrics.

2) **Dist($\mathbf{z}_1, \mathbf{z}_2$)**: Computes the Euclidian distance between two states $\mathbf{z}_1, \mathbf{z}_2 \in \mathcal{X}$.

3) **Cost($\mathbf{z}$)**: Computes the cost $c(\mathbf{z}) = c(\mathbf{z}_{parent}) + \text{Dist}(\mathbf{z}, \mathbf{z}_{parent})$ of a new node $\mathbf{z}$ as the cost of its parent $\mathbf{z}_{parent}$, plus the Euclidian distance $\text{Dist}(\mathbf{z}, \mathbf{z}_{parent})$.

4) **Nearest($\mathcal{T}, \mathbf{z}$)**: Given a tree $\mathcal{T}$ and a state $\mathbf{z} \in \mathcal{X}$, this function finds the nearest node $\mathbf{z}_{nearest} = \text{Nearest}(\mathcal{T}, \mathbf{z})$ in the tree by utilizing the $\text{Dist}(\cdot)$ function.

5) **Insert($\mathbf{z}_{current}, \mathbf{z}_{new}, \mathcal{T}$)**: Given the current node $\mathbf{z}_{current} \in \mathcal{X}$, the function inserts a new node $\mathbf{z}_{new}$ into the current tree $\mathcal{T}$ by adding it to $\mathcal{V}$, and connects it to $\mathbf{z}_{current}$ by adding the edge between them to $\mathcal{E}$.

6) **IsCollisionFree(σ)**: Checks whether a trajectory $\sigma : [0, T_{plan}] \rightarrow \mathcal{X}$ is outside the obstacle space $\mathcal{X}_{obst}$ for all $t \in \{0, T_{plan}\}$.

7) **Steer($\mathbf{z}_1, \mathbf{z}_2$)**: Computes a control input sequence that steers the own-ship from state $\mathbf{x}(0) = \mathbf{z}_1$ to $\mathbf{x}(T) = \mathbf{z}_2$ using the LOS method outlined in Section 2.3. The result is the final endpoint state $\mathbf{z}_{new}$ and corresponding trajectory $\sigma_{new} : [0, T] \rightarrow \mathcal{X}$ with steering horizon $T \in [T_{min}, T_{max}]$. The minimum and maximum steering time T_{min} and T_{max} are parameters to be adjusted based on the geography. Narrow channels and inland waterways might require lower steering times to avoid collisions, and vice versa for more open sea areas.

8) **ExtractBestSolution($\mathcal{T}$)**: Given the tree $\mathcal{T}$, the function extracts the solution

$$\sigma_d = \arg \min_{\sigma} (c(\sigma) \mid \sigma \in \mathcal{T} \wedge c(\sigma) < \infty), \quad (7)$$

if any. Otherwise, the algorithm will report failure. The solution trajectory is valid if the corresponding leaf node is inside an acceptance radius R_a of the goal $\mathbf{z}_{goal}$, and thus its cost is finite.

The Steer function enables the planner to consider the own-ship dynamics by using the motion model $f(\mathbf{x}(t), \mathbf{u}(t))$ for simulating a trajectory from a state $\mathbf{z}_1$ to $\mathbf{z}_2$. The speed and turn rate (course rate) are saturated to within the considered vessel minimum and maximum speeds $\{U_{min}, U_{max}\}$ and maximum turn rate r_{max} for this particular model, respectively.

To incorporate dynamic obstacle collision avoidance, one can employ a joint simulator as in [37] in the steering together with adding virtual obstacles for striving towards COLREG compliance, or utilize biased sampling methods as in e.g. [38]. However, this will not be considered in the present work.

For reductions in computational effort, we add the functionality that the planner attempts direct goal growth every Δ_{goal} iterations through the following function:

9) **DirectGoalGrowth($\mathcal{T}, \mathbf{z}_{goal}$)**: Finds the nearest neighbor $\mathbf{z}_{nearest} = \text{Nearest}(\mathcal{T}, \mathbf{z}_{goal})$ of the goal state, and attempts to steer the ship towards the state using $\text{Steer}(\mathbf{z}_{nearest}, \mathbf{z}_{goal})$ with a sufficiently large steering time, commonly a multiple of T_{max} . This returns a new node $\mathbf{z}_{new}$, which is added to the tree $\mathcal{T}$ if the steering is successful.

The baseline RRT algorithm can be described by these functions and is outlined in Algorithm 1. Here, N_{iter}^{max} is the

Algorithm 1 RRT.

Require: {Initial state z_{init} , goal state z_{goal} and ENC }

```

1:  $\mathcal{T} \leftarrow (\emptyset, \emptyset)$ 
2:  $\mathcal{T} \leftarrow \text{Insert}(\emptyset, z_{init}, \mathcal{T})$ 
3: for  $i = 1 : N_{iter}^{max}$  do
4:   if  $i \% \Delta_{goal} == 0$  then DirectGoalGrowth( $\mathcal{T}, z_{goal}$ )
5:   end if
6:    $z_{rand} \leftarrow \text{Sample}(i)$ 
7:    $z_{nearest} \leftarrow \text{Nearest}(\mathcal{T}, z_{rand})$ 
8:    $(z_{new}, \sigma_{new}) \leftarrow \text{Steer}(z_{nearest}, z_{rand})$ 
9:   if IsCollisionFree( $\sigma_{new}$ ) then
10:     $\mathcal{T} \leftarrow \text{Insert}(\mathcal{T}, z_{min}, z_{new})$ 
11:   end if
12: end for
13:  $\sigma_d \leftarrow \text{ExtractBestSolution}(\mathcal{T})$ 

```

maximum number of allowable iterations and Δ_{goal} the iterations between each attempt of DirectGoalGrowth. In addition to the maximum iteration constraint, we also under the hood put a constraint on the maximum number of nodes N_{node}^{max} allowable in the RRT planners and wall time t_{max} spent on optimizing the trajectory solutions.

3.3 RRT*

Specific to the RRT* variant are the NearestNeighbors, FindParent and Rewire functions, which give the algorithm probabilistic asymptotic convergence properties. The RRT* algorithm can be described fully by the functions 1-12 and is outlined in Algorithm 2.

10) NearestNeighbors: The function NearestNeighbors($\mathcal{T}, z, n$) = $\mathcal{V} \cap \text{Reachables}(z, l)$ finds the nearest neighbours $\mathcal{Z}_{near}$ in $\mathcal{V}$ of the state $z \in \mathcal{X}$, by calculation of the set $\text{Reachables}(z, l) = \{z' \in \mathcal{X} : \text{Dist}(z, z') \leq l \wedge \text{Dist}(z', z) \leq l\}$. The distance threshold is given as $l = l(n)$ and found such that the $\text{Reachables}(z, l)$ contains a ball with volume

Algorithm 2 RRT*.

Require: {Initial state z_{init} , goal state z_{goal} and ENC }

```

1:  $\mathcal{T} \leftarrow (\emptyset, \emptyset)$ 
2:  $\mathcal{T} \leftarrow \text{Insert}(\emptyset, z_{init}, \mathcal{T})$ 
3: for  $i = 1 : N_{iter}^{max}$  do
4:   if  $i \% \Delta_{goal} == 0$  then DirectGoalGrowth( $\mathcal{T}, z_{goal}$ )
5:   end if
6:    $z_{rand} \leftarrow \text{Sample}(i)$ 
7:    $z_{nearest} \leftarrow \text{Nearest}(\mathcal{T}, z_{rand})$ 
8:    $(z_{new}, \sigma_{new}) \leftarrow \text{Steer}(z_{nearest}, z_{rand})$ 
9:   if IsCollisionFree( $\sigma_{new}$ ) then
10:     $\mathcal{Z}_{near} \leftarrow \text{NearestNeighbors}(\mathcal{T}, z_{new})$ 
11:     $z_{min} \leftarrow \text{FindParent}(\mathcal{Z}_{near}, z_{nearest}, z_{new})$ 
12:     $\mathcal{T} \leftarrow \text{Insert}(\mathcal{T}, z_{min}, z_{new})$ 
13:     $\mathcal{T} \leftarrow \text{Rewire}(\mathcal{T}, z_{near}, z_{min}, z_{new})$ 
14:   end if
15: end for
16:  $\sigma_d \leftarrow \text{ExtractBestSolution}(\mathcal{T})$ 

```

$\gamma \log(n)/n$ for an appropriate parameter γ [7]. To reduce tree size and computation time, we enforce that the distance to the nearest node must satisfy $\text{Dist}(z_{new}, z_{nearest}) \geq d_{node}^{min}$, and consider a maximum number of neighbors of $N_{nn}^{max} = 10$ for all RRT* variants.

11) FindParent($\mathcal{Z}_{near}, z_{nearest}, z_{new}$): Finds the parent of the currently considered node $z_{new} \in \mathcal{X}$ that gives the minimal Cost(z_{new}) out of the nearest node set $\mathcal{Z}_{near}$ and the nearest node by distance $z_{nearest}$ [7].

12) Rewire($\mathcal{T}, \mathcal{Z}_{near}, z_{min}, z_{new}$): Checks if the tree $\mathcal{T}$ can be rewired to give a lower cost for z_{new} than the current parent z_{min} by connecting it to a minimal cost node in $\mathcal{Z}_{near}$. See [7] for more information.

Nearest neighbor extraction is a crucial part of RRT*, and is highly dependent on the search parameter γ , for which [14] provide some tuning considerations. We note that this parameter must be scaled based on the problem size. If dimensions > 2 are considered, with state entries of different units and scales, it will be important to normalize the states. For Euclidean distance-based search in two or three dimensions, this is, however, a necessity.

3.4 IRRT*

IRRT* varies from the baseline RRT* only in the fact that, once an initial solution is found with cost c_{best} , an admissible informed sampling heuristic is used onwards. The heuristic is formed from an elliptical domain given by the x_{start} and x_{goal} as focal points, the theoretical minimum cost $c_{min} = ||p_{goal} - p_{start}||_2$ and current best solution cost c_{best} , with p_{goal} and p_{start} being the planar position parts x_{start} and x_{goal} . It is shown that this heuristic enables *focused* planning towards x_{goal} , as opposed to the Voronoi bias property inherited in RRT and RRT* that lead to planning towards all points in the state space. The informed sample is drawn from a hyperellipsoid as follows:

$$z_{ell} = \mathbf{C}\mathbf{L}z_{ball} + z_{centre}, \quad (8)$$

with $z_{centre} = (x_{start} + x_{goal})/2$ and $\mathbf{L} \in \mathbb{R}^{n_x \times n_x}$ being the Cholesky decomposition of the matrix $\mathbf{S} = \mathbf{L}\mathbf{L}^T \in \mathbb{R}^{n_x \times n_x}$, which is given by

$$\mathbf{S} = \text{diag}\left\{\frac{c_{best}^2}{4}, \frac{c_{best}^2 - c_{min}^2}{4}, \dots, \frac{c_{best}^2 - c_{min}^2}{4}\right\}, \quad (9)$$

and which defines the ellipsoid

$$(z - z_{centre})^T \mathbf{S} (z - z_{centre}) \leq 1 \quad (10)$$

The rotation matrix $\mathbf{C}$ from the hyperellipsoidal frame to the world frame can be found by solving the Wahba problem [16].

Algorithm 3 IRRT*.

Require: {Initial state z_{init} , goal state z_{goal} and ENC}

```

1:  $\mathcal{T} \leftarrow (\emptyset, \emptyset)$ 
2:  $\mathcal{T} \leftarrow \text{Insert}(\emptyset, z_{init}, \mathcal{T})$ 
3: for  $i = 1 : N_{iter}^{max}$  do
4:   if  $i \% \Delta_{goal} == 0$  then DirectGoalGrowth( $\mathcal{T}, z_{goal}$ )
5:   end if
6:    $z_{rand} \leftarrow \text{InformedSample}(i)$ 
7:    $z_{nearest} \leftarrow \text{Nearest}(\mathcal{T}, z_{rand})$ 
8:    $(z_{new}, \sigma_{new}) \leftarrow \text{Steer}(z_{nearest}, z_{rand})$ 
9:   if IsCollisionFree( $\sigma_{new}$ ) then
10:     $\mathcal{Z}_{near} \leftarrow \text{NearestNeighbors}(\mathcal{T}, z_{new})$ 
11:     $z_{min} \leftarrow \text{FindParent}(\mathcal{Z}_{near}, z_{nearest}, z_{new})$ 
12:     $\mathcal{T} \leftarrow \text{Insert}(\mathcal{T}, z_{min}, z_{new})$ 
13:     $\mathcal{T} \leftarrow \text{Rewire}(\mathcal{T}, \mathcal{Z}_{near}, z_{min}, z_{new})$ 
14:   end if
15: end for
16:  $\sigma_d \leftarrow \text{ExtractBestSolution}(\mathcal{T})$ 

```

However, when sampling planar positions, it can be directly found as a 2D rotation matrix with angle $\theta_g = \text{atan2}(y_{goal} - y_{start}, x_{goal} - x_{start})$. The sampling is in this case reduced to

$$C_{2D} = \begin{bmatrix} \cos(\theta_g) & -\sin(\theta_g) \\ \sin(\theta_g) & \cos(\theta_g) \end{bmatrix}$$

$$p_{ball} \sim \text{Uniform}(p; \mathbf{0}_{2 \times 1}, \mathbf{1}_{2 \times 1}) \quad (11)$$

$$p_{ell} = C_{2D} L_{2D} p_{ball} + p_{centre}$$

$$z_{ell} = [p_{ell}^T, 0, 0]^T,$$

where $\text{Uniform}(x; a, b)$ is a uniform distribution with interval limits a and b . The informed sampling procedure is described below, with the IRRT*-variant being described in Algorithm 3.

13) InformedSample(i): Given the current best cost c_{best} , sample a new state using Eq. 11. If $c_{best} = \infty$, use the baseline Sample(i).

3.5 PQ-RRT*

The Potential-Quick RRT* combines the features of P-RRT* and Q-RRT* [17], which involves sample adjustments using a goal-based potential field attractive force, and ancestor consideration in the tree growth for path length reduction, respectively. The potential-field based sample adjustment procedure is shown in Algorithm 4, whereas the Ancestry functionality are described by the following procedures:

14) Ancestor($\mathcal{T}, z, \phi$): Given the tree $\mathcal{T}$, a node z and depth parameter $\phi > 0$, the ϕ -th parent of z is returned. If the tree is not deep enough, no ancestor is returned.

15) Ancestry($\mathcal{T}, \mathcal{Z}$): Given the tree $\mathcal{T}$ and node set $\mathcal{Z}$, it returns $\mathcal{Z}_{ancestors} = \bigcup_{z \in \mathcal{Z}} \left[\bigcup_{l=1}^d \text{Ancestor}(\mathcal{T}, z, l) \right]$, and $\emptyset$ if the tree depth is 0.

Algorithm 4 AdjustSample.

Require: {Sampled state z_{rand} , goal state z_{goal} }

```

1:  $z_{prand} \leftarrow z_{rand}$ 
2: for  $k = 1 : N_{sa}^{max}$  do
3:    $F_{att} \leftarrow (z_{goal} - z_{prand})$ 
4:    $d_{min} \leftarrow \text{DistanceNearestObstacle}(\mathcal{X}_{obst}, z_{prand})$ 
5:   if  $d_{min} < d_{margin}$  then
6:     return  $z_{prand}$ 
7:   else
8:      $z_{prand} \leftarrow z_{prand} + \lambda \frac{F_{att}}{\|F_{att}\|}$ 
9:   end if
10: end for
11: return  $z_{prand}$ 

```

16) PQRewire($\mathcal{T}, \mathcal{Z}_{near}, z_{min}, z_{new}$): Same as Rewire, except from the fact that the ancestry of z_{new} is also considered as a rewiring node.

The PQ-RRT* is described in Algorithm 5. In this work, we consider a depth-level of $\phi = N_{ancestry}$ in the Ancestry procedure, whereas a constant depth level of 1 is used in the PQRewire method, to reduce computational effort. Note that the algorithm as proposed in [17] was not tested with kinodynamical constraints and motion planning in the steering, which will induce substantially higher run-times in the algorithm due to the ancestor consideration in the FindParent and PQ-Rewire routines. On the other hand, better convergence properties are expected due to the PQ features. The sample adjustment procedure uses a hazard clearance parameter d_{margin} that must be set based on acceptable margins, the ship type, and possibly other factors. We note that the selection of the PQ-RRT* specific parameters is non-trivial, and no guidelines for their selection were provided in [17]. The authors highlighted this as a potential limitation of the method.

Algorithm 5 PQ-RRT*.

Require: {Initial state z_{init} , goal state z_{goal} and ENC}

```

1:  $\mathcal{T} \leftarrow (\emptyset, \emptyset)$ 
2:  $\mathcal{T} \leftarrow \text{Insert}(\emptyset, z_{init}, \mathcal{T})$ 
3: for  $i = 1 : N_{iter}^{max}$  do
4:   if  $i \% \Delta_{goal} == 0$  then DirectGoalGrowth( $\mathcal{T}, z_{goal}$ )
5:   end if
6:    $z_{rand} \leftarrow \text{Sample}(i)$ 
7:    $z_{rand} \leftarrow \text{AdjustSample}(z_{rand}, z_{goal})$ 
8:    $z_{nearest} \leftarrow \text{Nearest}(\mathcal{T}, z_{rand})$ 
9:    $(z_{new}, \sigma_{new}) \leftarrow \text{Steer}(z_{nearest}, z_{rand})$ 
10:  if IsCollisionFree( $\sigma_{new}$ ) then
11:     $\mathcal{Z}_{near} \leftarrow \text{NearestNeighbors}(\mathcal{T}, z_{new})$ 
12:     $\mathcal{Z}_{ancestors} \leftarrow \text{Ancestry}(\mathcal{T}, \mathcal{Z}_{near})$ 
13:     $\mathcal{Z}_{union} \leftarrow \mathcal{Z}_{near} \cup \mathcal{Z}_{ancestors}$ 
14:     $z_{min} \leftarrow \text{FindParent}(\mathcal{Z}_{union}, z_{nearest}, z_{new})$ 
15:     $\mathcal{T} \leftarrow \text{Insert}(\mathcal{T}, z_{min}, z_{new})$ 
16:     $\mathcal{T} \leftarrow \text{PQRewire}(\mathcal{T}, \mathcal{Z}_{near}, z_{min}, z_{new})$ 
17:  end if
18: end for
19:  $\sigma_d \leftarrow \text{ExtractBestSolution}(\mathcal{T})$ 

```

4 Practical Aspects in RRT-based Planning

4.1 Sampling

One of the most important parts of RRT is the method by which new states are sampled. Brute force sampling of states $\mathbf{z}_{rand} \in \mathcal{X}_{free}$ is not recommended, as the rejection rate will be proportional to the volume of $\mathcal{X}_{obs}$ relative to the total space $\mathcal{X}$. Instead, one can for instance create and sample from a Constrained Delaunay Triangulation (CDT), made from the safe sea area that the ship is to voyage within, similar to [39]. This gives a set Θ_{tri} of triangles with indices $j = 1, 2, \dots, n_{tri}$, which in this application reduces the sampling of a new state position $\mathbf{p}_{rand} \in \mathbb{R}^2$ to

$$\begin{aligned} j' &\sim \text{WeightedUniform}(j; 1, n_{tri}) \\ r_1 &\sim \text{Uniform}(r; 0, 1) \quad r_2 \sim \text{Uniform}(r; 0, 1) \\ \mathbf{p}_{rand} &= (1 - \sqrt{r_1})\mathbf{A}_{j'} + \sqrt{r_1}(1 - r_2)\mathbf{B}_{j'} + \sqrt{r_1}r_2\mathbf{C}_{j'} \end{aligned} \quad (12)$$

where $(\mathbf{A}_{j'}, \mathbf{B}_{j'}, \mathbf{C}_{j'})$ are the vertices of triangle j' , and WeightedUniform is a uniform distribution weighted by the area of each triangle. The random state is then found as $\mathbf{z}_{rand} = [\mathbf{p}_{rand}^T, 0, 0]^T$. An illustration of a CDT of the safe sea area used for sampling in the second planning example is shown in Fig. 1.

Fig. 1 Constrained Delaunay Triangulation for a considered planning area near Stavanger, Norway. Triangles are shown with green edges, Land in dark blue, whereas the shore and seabed are shown in variations of brighter blue



When using the sample heuristic in IRRT* bonafide, the same problem of high rejection rates can occur. Here, one can again use CDT, and prune triangles outside the ellipsoidal sampling domain after a new solution is found and the heuristic is put to use. However, this will not be done here.

4.2 Data Structures

Another foundation of the RRT algorithms is nearest neighbor searches, used when wiring and re-wiring the tree for RRT*-variants. For spatial queries as is considered here, it is recommended to use R*-tree or R-tree-based data structures [40, 41] that are commonly used for storing spatial objects in e.g. databases. They have $O(\log(n))$ complexity for insertion and distance-related queries for n inserted elements. The trees are created such that leaf nodes of the tree hold spatial data, and parents or branching nodes correspond to the minimum bounding box that contains all of its children. With this structure, the R-tree utilizes data-based partitions into boxes of decreasing size as the tree grows. K-d trees have the same complexity as R-trees, and can also be used [42]. However, note that the bonafide version is suitable only when the nodes are points and does not work well in high dimensions. Once polygons and other objects are introduced for representing the own-ship, static or dynamic hazards, standard k-d trees are also not compatible. The advantages of using R-trees are the efficient memory layout, tree update procedures and

spatial nearest neighbor searching. For cases where the tree will not end up with a high number of objects or when few modifications and removals will be made on the go, it can on the other hand be sufficient to use k-d trees [42].

These points also apply to the collision checking part, discussed below. Note that it might be worthwhile to consider a Mahalanobis distance metric when it is desired to evaluate the distance with respect to both position and other state variables such as orientation. In this regard, it will be wise to normalize the data before considering distance calculations.

4.3 Collision Checking

To accept a new node in the tree $\mathcal{T}$ maintained by the RRT algorithms, collision checks must be performed for each new trajectory segment σ_{new} resulting from the steering function in several parts of the RRT-algorithms. To ensure feasibility with respect to run-time, it is important to pre-process grounding hazard data using line simplification algorithms such as Ramer-Douglas-Peucker [43] to reduce the map data accuracy to the required level. Especially for the maritime domain, one should merge multi polygons arising from hazardous *land*, *shore* and *seabed* objects extracted from Electronic Navigation Charts (ENC) [44], given the considered vessel and its draft. We also recommend removing interior holes of land multi polygons arising from e.g. lakes, as these will naturally not be considered for sea voyages.

Again, it will also be important in this procedure to use efficient data structures such as R-trees for enabling fast distance computations in collision checking. Adherence to specified hazard clearance margins d_{safe} and control requirements can easily be done by buffering the hazard polygons before running the algorithm. Note that verification of sampled poses along a trajectory being collision-free is not exact, and operation in more confined space with smaller distance margins might require more accurate methods as in e.g. Zhang et al. [45].

Note that collision checking requires significantly less effort when only two-point planar waypoint segments are considered, as opposed to a full trajectory segment with kinodynamical constraint consideration. This can be an option when the goal is to rapidly generate waypoints for a ship to follow, as opposed to a full trajectory to track. Some regard to maximum curvature based on the ship turn rate and speed should then be taken into account when accepting waypoint segments.

4.4 Steering

For trajectory planning RRTs, the steering functionality for connecting state pairs (z_1, z_2) is a crucial part of the tree wiring, especially for non-holonomic systems. Previous methods have proposed e.g. the simplistic Dubin's path for optimal steering [7], Bezier spline-based connectivity [46],

learning-based steering and collision checking [21], LQR-control-based extension [47] and lazy steering with a Neural Network (NN) for determining collision-free and steerable node extensions [48]. In this work, we apply LOS guidance for steering the ship towards new eligible nodes, as a low-cost solution for creating feasible trajectory segments in the maritime domain. One point that requires care in this context, is the early termination of the LOS guidance if the waypoint segment from z_1 to a new node z_2 is passed by, such that the computational cost of numerical integration of the ship dynamics is kept to a minimum. Further note that the integration time step should be increased in tact with the problem size, also related to the computational cost aspect.

5 Results

We present three cases in which the considered RRT-variants are compared. The first case demonstrates the usage of RRTs for rapid ship behavior generation, where the goal is to generate kinodynamically feasible trajectories for a specified ship scenario. The second case compares common RRT variants for a smaller planning scenario with a typical local minimum problem. Finally, the third case compares the RRT planners in a larger planning scenario. The simulation framework in [49] is used as a platform for developing and testing the planners, which allows for the utilization of Electronic Navigational Charts. The algorithms are implemented in the *Rust programming language*, and the executable is run on a MacBook Pro with an Apple Silicon M1 chip. We evaluate the obtained solutions with respect to run-time and trajectory lengths, over $N_{MC} = 100$ Monte Carlo (MC) simulations for each case, where the RRTs are provided with different pseudorandom seeds before each run. Aside from summarizing the planner results, we report the statistical significance of the trajectory length results produced by PQ-RRT* being more distance optimal than other RRT variants or not through Welch's unequal variance Student's t -test [50].

The criteria for the planners to terminate in the case studies in Sections 5.2 - 5.3 are common and entails that one of the following conditions are true:

- The maximum tree node number budget N_{node}^{max} is met.
- The maximum iteration number budget N_{iter}^{max} is met.
- The maximum wall time budget t_{max} is met.

The parameter values for the above termination criteria are given later in this section. The intuition behind also using a maximum tree node or size number is to limit the memory consumption of the planners. Also, it is good to have a time budget since we can then measure the solution quality attainable within a given set timespan, whereas having only an iteration number termination criteria could mean a

very large spread in run-times depending on the algorithm. In practice, we put a high maximum tree node number and iteration number, such that the time budget will be the most constraining factor.

A kinematic model with parameters $T_X = 6$ s, $T_U = 6$ s, $r_{max} = 10$ °/s, $U_{min} = 0$ knots and $U_{max} = 20$ knots is considered for a ship with length about 15 m, with LOS guidance parameters $\Delta = 30.0$ m.

The grounding hazards in the environment are buffered with a horizontal clearance parameter d_{safe} of 0 m in the first two cases, and 5 m in the last case. This will in general be dependent on the map accuracy, ship type, and application. We consider a time step δ_{sim} of 0.5 s in the first two cases, and 1.0 s in the last case, for the RRT kinematic ship model. For the RRTs, we use R-trees to perform nearest neighbor searching and spatial queries, where grounding hazards (land, shore, and relevant seabed) are extracted and merged considering a vessel draft of 1.0 m. The merged hazards are then used to form a safe sea CDT, from which weighted samples are drawn.

Key parameters for the RRT-variants used in the first two cases are given in Table 1. For the first case, we did not find a configuration of PQ-RRT* parameters for the sample adjustment procedure that gave a better result than just disabling the APF-based part as a whole and thus used $N_{sa}^{max} = 0$. All methods sample from the safe sea CDT using Eq. 12, except the IRRT* which uses Eq. 8 after a solution has been found. The PQ-RRT* will adjust the CDT sample using its goal potential field. We note that tuning of the RRT algorithms is non-trivial, and will be specific to the scale considered for the trajectory planning. This is a trade-off between planner run-time, memory requirements, and solution quality. The number of iterations should be adequately high to increase the likelihood of the RRT finding a solution.

Table 1 Algorithm parameters for the smaller planning cases

Parameter	PQ-RRT*	IRRT*	RRT*	RRT
N_{node}^{max}	10000	10000	10000	10000
N_{iter}^{max}	25000	25000	25000	25000
Δ_{goal}	500	500	500	500
d_{node}^{min}	5.0 m	5.0 m	5.0 m	—
γ	2000.0 m	2000.0 m	2000.0 m	—
T_{min}	1.0 s	1.0 s	1.0 s	1.0 s
T_{max}	30.0 s	30.0 s	30.0 s	30.0 s
R_d	10.0 m	10.0 m	10.0 m	10.0 m
δ_{sim}	0.5 s	0.5 s	0.5 s	0.5 s
d_{margin}	0.1 m	—	—	—
$N_{ancestry}$	1	—	—	—
N_{sa}^{max}	0	—	—	—
λ_{sample}	1.0 m	—	—	—

5.1 Random Vessel Trajectory Generation

The first case is a random vessel trajectory generation scenario where the goal is to generate multiple random trajectories from a given start position. After being built, the resulting RRT variant can be queried efficiently through R-tree spatial nearest neighbor search [40], and used to rapidly sample random ship trajectories for intention-aware CAS or simulation-based testing of CAS. This is useful in the online setting for CAS, but also for interaction data generation to be used by learning-based CAS algorithms. Training of Reinforcement Learning (RL) agents with the [Gymnasium](#) framework typically requires a reset of the environment after each terminated or truncated episode. In this context, RRT algorithms can be used for rapidly generating target ship trajectory scenarios after a reset. As an example, PQ-RRT* will, in general, produce more path-optimal solutions than RRT and RRT*, and can thus be used for spawning target ship behavior scenarios with minimal maneuvering, whereas the RRT* or RRT variants can be used for spawning gradually unpredictable maneuvering target ship behaviors that can be considered as outliers. Thus, employing multiple RRT algorithm variants for ship scenario generation can improve the ability of learning-based CAS to generalize, and also enlarge the test coverage in the context of simulation-based CAS verification.

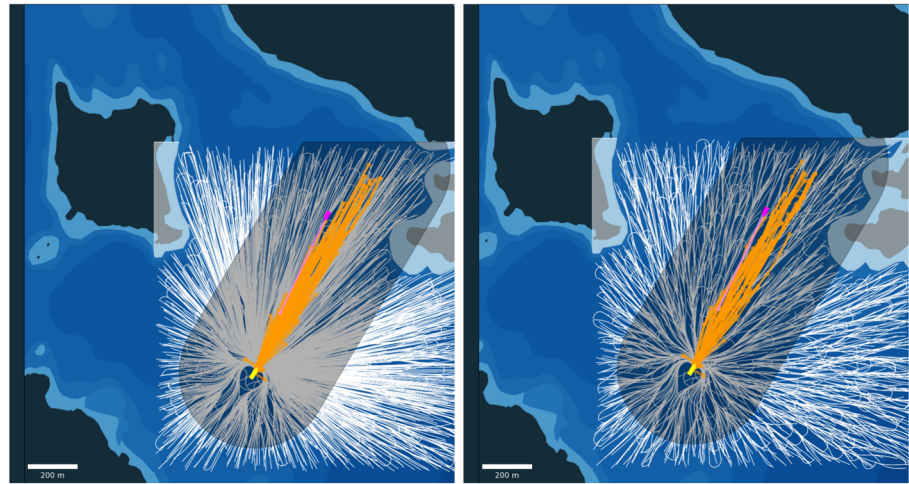
Note that, in the scenario-generation context the RRT cost function can be designed to e.g. minimize time to collision or converge towards near misses between the random vessel and the own-ship that runs the CAS to be tested [26]. Furthermore, in the maritime context, COLREG can be misinterpreted and lead to ambiguous and therefore often dangerous situations [51]. Thus, RRTs could be guided towards edge case situations in COLREG where the applicable situation rule(s) are easy to misinterpret. These considerations are a topic for future work.

Figure 2 shows an example of obstacle ship behavior generation for a CAS head-on situation. To make the figures less dense, we have reduced the maximum allowable tree nodes to $N_{node}^{max} = 4000$ for all planners. In this particular case, we find built RRT, RRT*, and PQ-RRT* behaviors close to a randomly sampled position within a corridor given by the initial own-ship position and course and its maximum travel length over the simulation timespan, i.e.

$$\begin{aligned}
 x_{corr} &\sim \text{Uniform}(x; 0, U_d^{os} T_{sim}) \\
 y_{corr} &\sim \text{Uniform}(y; -d_{corr}/2, d_{corr}/2) \\
 \theta_{corr} &= \text{atan2}(y_{end}^{os} - y_{start}^{os}, x_{end}^{os} - x_{start}^{os}) \\
 p_{rand} &= p_{start}^{os} + R(\theta_{corr}) \begin{bmatrix} x_{corr} \\ y_{corr} \end{bmatrix},
 \end{aligned} \tag{13}$$

with d_{corr} as the corridor width parameter, T_{sim} as the total simulation time-span, U_d^{os} is the own-ship speed reference,

Fig. 2 Results from random obstacle ship behavior generation for a CAS head-on situation near Halsnøya in Boknafjorden, Norway. The initial obstacle ship position is shown in yellow, the initial own-ship position in pink and its waypoints in purple, the resulting tree in white, sampled obstacle ship behavior waypoint sequences in connected orange annuluses. The planning bounding box is shown in transparent white



(a) PQ-RRT*.

(b) RRT*.



(c) RRT.

and $\mathbf{p}_{start}^{os} = [x_{start}^{os}, y_{start}^{os}]^T$ and $(x_{end}^{os}, y_{end}^{os})$ are the own-ship start and end coordinates, respectively.

Figure 3 shows another case where we sample behaviors for a CAS crossing situation. In this situation, random position samples are drawn near the predicted closest point of approach (CPA) between the vessels, from which the nearest RRT behaviors or trajectories are fetched, i.e.

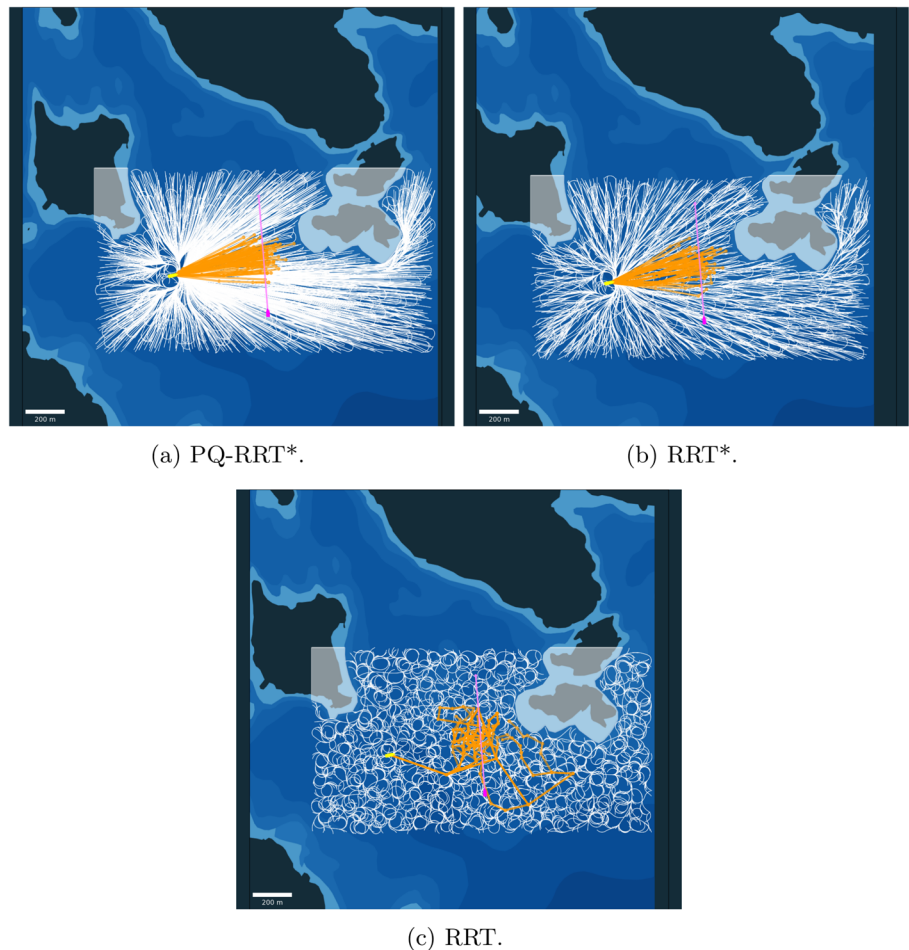
$$\mathbf{p}_{rand} = \text{Normal}(\mathbf{p}; \mathbf{p}_{cpa}, \mathbf{\Sigma}), \quad (14)$$

where $\mathbf{p}_{cpa}$ is the target ship position at CPA, assuming constant speed and course for the two vessels, calculated as in e.g. Kuwata et al. [52], and where $\mathbf{\Sigma}$ is the covariance parameter adjusting the spread of samples. This strategy allows for generating multiple target ship behaviors that will lead to a collision or near miss with the own-ship unless preventive actions are taken. Since the RRTs are flexible, any of the navigational risk factors as outlined in Bolbot et al. [31] could

be considered as targets for developing either RRT sampling schemes or cost functions.

For each of the example situations, the sampled behaviors parameterized by waypoints are shown. Since the planners consider the underlying ship model dynamics, the waypoints will be feasible, in addition to being collision-free with respect to nearby static hazards. Note that we can also use the trajectory that accompanies the waypoints as well. Further note that we define a reduced-size bounding box considered by the RRT planners, in order to reduce computation time and consider only a subset of the grounding hazards present in the ENC. Once the RRTs are built, a new behavior can be sampled in less than 1ms on the considered computing platform, which makes the approach viable for large-scale scenario production. We also note that the trees can easily be built offline, and then effectively sampled from afterwards in the relevant context.

Fig. 3 Results from random obstacle ship behavior generation for a CAS crossing situation near Børøy in Boknafjorden, Norway. The initial obstacle ship position is shown in yellow, the initial own-ship position in pink and its waypoints in purple, the resulting RRT in white, sampled obstacle ship behavior waypoint sequences in connected orange annuluses. The planning bounding box is shown in transparent white



5.2 Smaller Planning Example

In the second case, we consider a smaller planning situation with a constant reference ship speed $U_d = 4.0$ m/s, and allow the planners to find and refine their solution over the maximum allowable iterations and tree nodes up until a maximum time of $t_{max} = 50$ s. The considered location near Kvitsøy in Rogaland, Norway has a map size of $850 \text{ m} \times 750 \text{ m}$, and features a typical local minimum problem where one can get stuck in the dead end not far from the ship initial position. Note that as the planners sample from a CDT constructed from the safe sea area, the sampling efficiency will be higher, making it easier to avoid local minima issues.

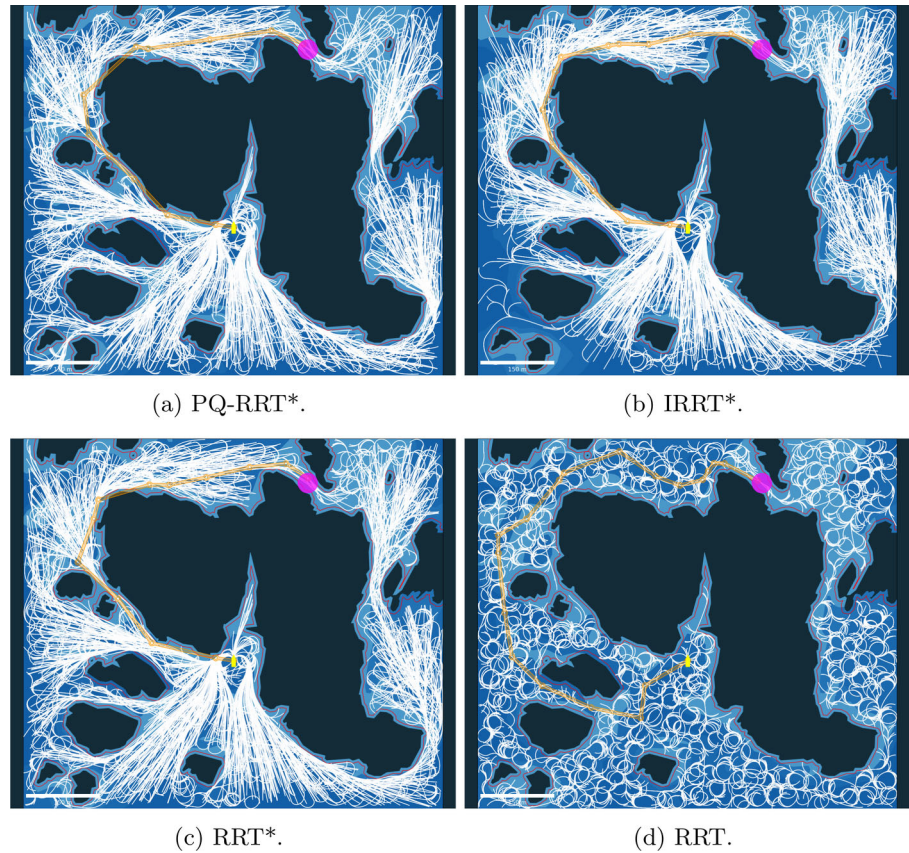
Results from sample runs are shown in Fig. 4, whereas solution statistics are given in Table 2. In the table, statistics for the time to find an initial solution $t_{sol,0}$ are reported, whereas run-time t_{sol} and path length d_{sol} statistics are reported for the final refined trajectory. The metric ρ_{mc} is the success percentage of finding a solution out of all the MC runs. From Fig. 4, it might look like there is a collision due to the orange waypoints crossing a hazard at some point. This is not the case, as the actual ship trajectories wired by the

RRTs are collision-free, taking the nonholonomic properties of the ship into account.

The optimal solution has a path length of approximately 905 m. Thus we see that PQ-RRT*, IRRT*, and RRT* can converge to within 6% of the optimum. PQ-RRT* attains the best results concerning path length, with a marginal difference to IRRT* and RRT*. On the other hand, the ancestor consideration in the tree rewiring and extra sampling functionality comes at the cost of higher runtimes. We also see a factor of 10 increase in the run-time between baseline RRT and RRT*, which is expected due to the more complicated wiring process.

For IRRT* we can get marginally better results than RRT* at lower run-times. This is due to the high sample rejection rate after finding an initial solution, as samples are likely to be taken from $\mathcal{X}_{obst}$ in this map with large hazard coverage. However, accepted samples will have higher values, partially explaining the marginal improvement in solution lengths. It can here be a viable approach to iteratively prune out triangles in the safe sea CDT as the hyperellipsoid forming the sampling heuristic reduces in volume. One should thus implement efficient methods for re-computing the CDT from

Fig. 4 Results from the first MC sample run on the smaller planning case near Kvitsøy outside Stavanger, Norway. The starting ship pose is shown in yellow, the resulting tree in white, node waypoints in orange, and the goal in purple. The safety buffered hazard boundary is shown in red



the new sampling domain or for pruning CDT triangles outside the domain.

The statistical significance of the results is gauged by using Welch's t -test [50], considering the null hypothesis H_0 that the mean Euclidean trajectory length from PQ-RRT* is less than that of all the other RRT planners, and the alternative hypothesis H_1 that the trajectory length of PQ-RRT* is equal to or larger than the other RRT planners:

$$\begin{aligned} H_0 : \mu_{p,\text{PQ-RRT}^*} - \mu_{p,i} &< 0, & i &\in \{\text{RRT}, \text{RRT}^*, \text{IRRT}^*\} \\ H_1 : \mu_{p,\text{PQ-RRT}^*} - \mu_{p,i} &\geq 0, & i &\in \{\text{RRT}, \text{RRT}^*, \text{IRRT}^*\} \end{aligned} \quad (15)$$

where $\mu_{p,\text{PQ-RRT}^*}$ is the mean trajectory length produced by the PQ-RRT* planner (equal to 963.1 m in the smaller planning example) and $\mu_{p,i}$ the mean trajectory length for the other RRT planners.

We start by computing Welch's test statistic from the sampled data as

$$t_{s,i} = \frac{\mu_{p,\text{PQ-RRT}^*} - \mu_{p,i}}{s_i} \quad (16)$$

which follows Student's t -distribution under the null hypothesis H_0 , where the pooled standard deviation s_i is computed

Table 2 Results for the smaller planning case over $N_{mc} = 100$ MC runs

Metric	PQ-RRT*	IRRT*	RRT*	RRT
$t_{sol,0}$	5.5 ± 5.0 s	3.5 ± 3.4 s	2.9 ± 1.8 s	0.14 ± 0.07 s
$(t_{sol,0}^{min}, t_{sol,0}^{max})$	(0.2 s, 29.1 s)	(0.2 s, 15.4 s)	(0.5 s, 11.2 s)	(0.03 s, 0.45 s)
t_{sol}	34.6 ± 2.5 s	26.3 ± 0.7 s	32.6 ± 0.5 s	1.91 ± 0.03 s
$(t_{sol}^{min}, t_{sol}^{max})$	(28.8 s, 38.9 s)	(24.6 s, 28.8 s)	(31.5 s, 34.1 s)	(1.8 s, 2.0 s)
d_{sol}	963.1 ± 32.0 m	965.9 ± 31.0 m	968.3 ± 33.3 m	1472.7 ± 163.7 m
$(d_{sol}^{min}, d_{sol}^{max})$	(914.0 m, 1081.9 m)	(920.0 m, 1072.0 m)	(909.9 m, 1070.0 m)	(1200.0 m, 2013.8 m)
ρ_{mc}	100%	100%	100%	100%

as

$$s_i = \frac{1}{\sqrt{N_{MC}}} \sqrt{\varepsilon_{p, \text{PQ-RRT}^*}^2 + \varepsilon_{p,i}^2}, \quad i \in \{\text{RRT}, \text{RRT}^*, \text{IRRT}^*\} \quad (17)$$

To check whether the null hypothesis H_0 holds, we use a significance level of $\alpha = 0.05$ and compute the p -values $P(t_{s,i} \geq t_{1-\alpha,i} | H_0)$ under the null hypothesis for the observed test statistics. This is done by utilizing the cumulative Student's t -distribution function with $N_{DOF,i}$ degrees of freedom, computed through the Welch-Satterthwaite equation [50]

$$N_{DOF,i} = N_{MC}^2 (N_{MC} - 1) \frac{s_i^4}{\varepsilon_{p, \text{PQ-RRT}^*}^4 + \varepsilon_{p,i}^4}, \quad (18)$$

which is valid for the case when the sample sizes of the two means are equal. Here, ε_i is the standard deviation of the trajectory length for the relevant planner (equal to 32.0 m in the smaller planning example for PQ-RRT*). Then, the threshold value used to determine whether we reject the null hypothesis or not is given by $t_{1-\alpha,i} = \text{tDistributionCDF}(1-\alpha, N_{DOF,i})$, where tDistributionCDF is the Student's t cumulative distribution function. The results are summarized in Table 3, and show that the null hypothesis holds under the significance level of $\alpha = 0.05$ since all the p -values are non-significant ($> \alpha$).

5.3 Larger Planning Example

The larger planning example considers the region shown in Fig. 1 of size 5.0 km $\times$ 5.0 km with multiple islands and smaller grounding hazards, which increases the problem size substantially. Again, the planners are allowed to find and refine a solution over the maximum allowable iterations and nodes, up until a maximum time of 300 s. A constant reference speed $U_d = 5.0$ m/s is utilized. In this case, due to the large map size, we utilize planner parameters as in Table 4.

Visual results when applying the RRT-variants on the planning area in Fig. 1 are shown for sample runs in Fig. 5. Table 5 shows performance metrics for the algorithms over the MC

Table 4 Algorithm parameters for the larger planning case

Parameter	PQ-RRT*	Informed-RRT*	RRT*	RRT
N_{node}^{max}	10000	10000	10000	10000
N_{iter}^{max}	25000	25000	25000	25000
Δ_{goal}	500	500	500	500
d_{node}^{min}	15.0 m	15.0 m	15.0 m	—
γ	3500.0 m	3500.0 m	3500.0 m	—
T_{min}	1.0 s	1.0 s	1.0 s	1.0 s
T_{max}	30.0 s	30.0 s	30.0 s	30.0 s
R_a	10.0 m	10.0 m	10.0 m	10.0 m
δ_{sim}	1.0 s	1.0 s	1.0 s	1.0 s
d_{margin}	0.5 m	—	—	—
$N_{ancestry}$	1	—	—	—
N_{sa}^{max}	50	—	—	—
λ_{sample}	8.0 m	—	—	—

runs. Again, we see a marginally better result for the PQ-RRT* than for IRRT* and RRT*. In this planning example, IRRT* achieves worse results than RRT* due to the significantly higher sample rejection rate again caused by a large obstacle congestion ratio, amplified by the problem scale.

The optimal solution is approximately 5.2 km, and thus we see that the optimal algorithm variants only converge to within approximately 30% of the optimum. The convergence issue is attributed to the map size, the optimal solution passing through narrow passages, and the complexity of considering ship dynamics and kinodynamical constraints in the tree wiring. A standard deviation of over 500 m is found for the path length solutions of all variants, which is significantly high.

Again we check the statistical significance of PQ-RRT* giving more distance-optimal trajectories than the other variants, by using a one-sided Welch's t -test as in the previous section, with the hypotheses in Eq. 15. The results are summarized in Table 6, and again it is shown that the null hypothesis holds under the significance level of $\alpha = 0.05$ since all the p -values are non-significant.

5.4 Discussion

5.4.1 Problem Complexity and Scalability

Gauging the results on planning, we see that the RRT-based planners are viable for use in problems of adequate size, i.e. less than 1 km $\times$ 1 km in map size. In these cases, the planners can find and optimize the best solution in adequate time. However, for larger maps, the planners struggle. This is attributed to the increased number of samples and iterations required in order to find and refine a solution. Although the planners find initial solutions fast, they have

Table 3 Hypothesis testing results for the trajectory length difference of PQ-RRT* versus the other planners in the smaller planning case

	$i = \text{IRRT}^*$	$i = \text{RRT}^*$	$i = \text{RRT}$
$t_{s,i}$	-0.6285	-1.1260	-30.5519
s_i	4.4553	4.6183	16.6798
$N_{DOF,i}$	197	197	106
$t_{1-\alpha,i}$	1.6526	1.6526	1.6526
p -value	0.7348	0.8692	1.0

Fig. 5 Results from the first MC sample run on the larger planning case near Vågen in Stavanger, Norway. The starting ship pose is shown in yellow, the resulting tree in white, node waypoints in orange, and the goal in purple. The safety buffered hazard boundary is shown in red

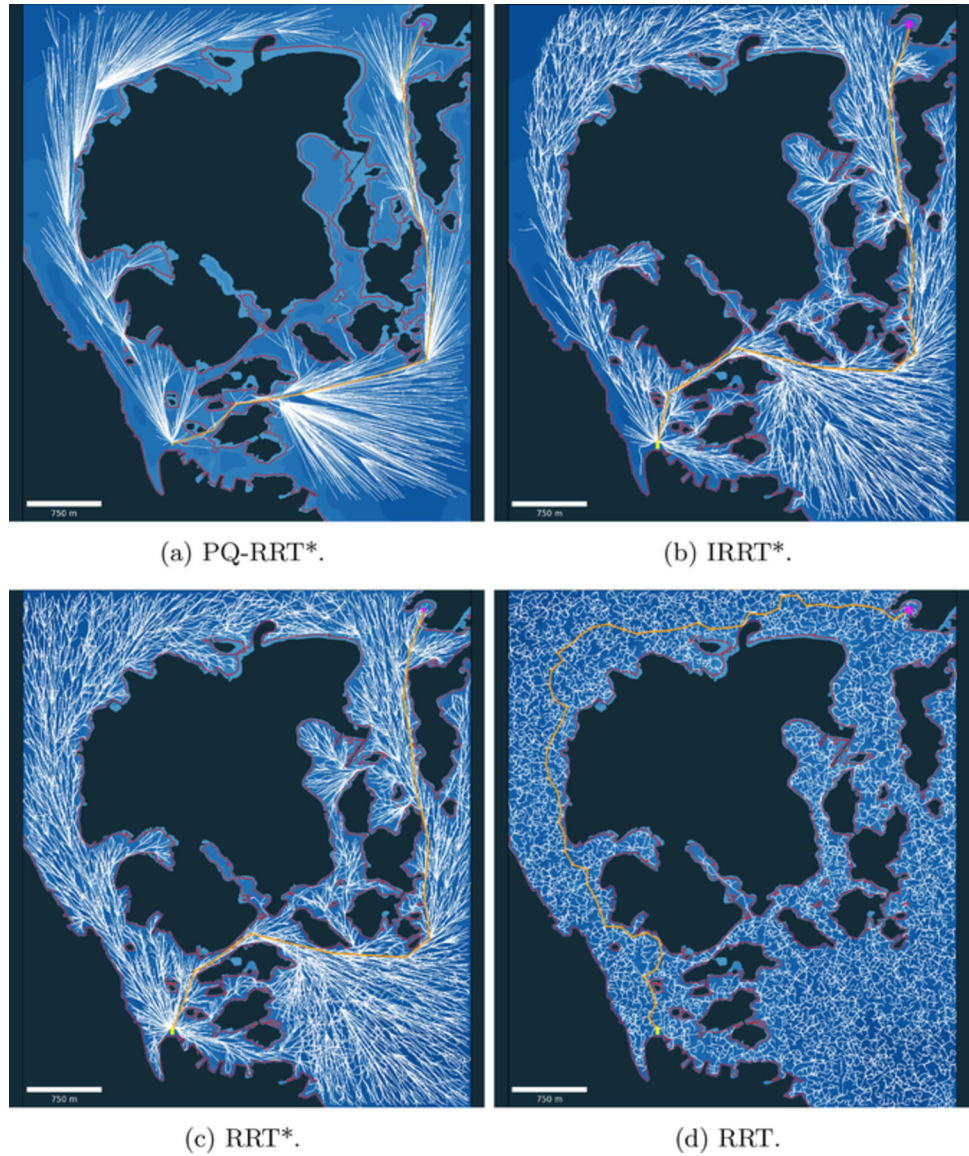


Table 5 Results for the larger planning case over $N_{mc} = 100$ MC runs

Metric	PQ-RRT*	IRRT*	RRT*	RRT
$t_{sol,0}$	62.7 ± 22.3 s	60.7 ± 31.0 s	60.7 ± 31.0 s	3.8 ± 1.4 s
$(t_{sol,0}^{min}, t_{sol,0}^{max})$	(33.7 s, 162.5 s)	(19.0 s, 205.0 s)	(19.0 s, 205.0 s)	(1.6 s, 9.6 s)
t_{sol}	300.0 ± 0.0 s	300.0 ± 0.0 s	300.0 ± 0.0 s	24.9 ± 2.1 s
$(t_{sol}^{min}, t_{sol}^{max})$	(300.0 s, 300.0 s)	(300.0 s, 300.0 s)	(300.0 s, 300.0 s)	(21.6 s, 27.5 s)
d_{sol}	6567.3 ± 407.7 m	6836.6 ± 476.6 m	6744.6 ± 487.4 m	8805.3 ± 620.8 m
$(d_{sol}^{min}, d_{sol}^{max})$	(5239.9 m, 7359.9 m)	(5299.9 m, 7534.9 m)	(5269.9 m, 7440.0 m)	(6484.0 m, 10178.5 m)
ρ_{mc}	100%	100%	100%	100%

Table 6 Hypothesis testing results for the trajectory length difference of PQ-RRT* versus the other planners in the larger planning case

	$i = \text{IRRT}^*$	$i = \text{RRT}^*$	$i = \text{RRT}$
$t_{s,i}$	-4.2938	-2.7902	-30.1331
s_i	62.7190	63.5435	74.2706
$N_{DOF,i}$	193	192	171
$t_{1-\alpha,i}$	1.6528	1.6528	1.6538
p -value	0.9999	0.9971	1.0

a hard time optimizing the solutions when considering larger map sizes and complex environments. The run-time also increases substantially for larger problem sizes, due to the higher computational cost of re-wiring the tree and propagating new node costs to the leaves.

Thus, in such large cases, it can be an option to utilize RRT-based planners without dynamics consideration, where the tree wiring only considers position sampling and the connection of these through collision-free straight-line segments. Alternatively, one can use path complete algorithms such as A*, to find the solution fast up to a suitable grid resolution, without having randomness in the result. Note that it will then be necessary to post-process the solution to generate a feasible trajectory for the ship to track, unless a motion primitives-based planner is used.

We note that IRRT*, although providing linear algorithm convergence properties for obstacle-free environments, struggles with both planning cases and especially the larger one. This is due to the large volume occupied by obstacles in the configuration space, leading to a significant rejection rate in the informed heuristical sampling. Thus, for IRRT* to be of practical usage, it requires an improvement. This can be an update and creation of a new CDT for the safe sea domain inside the informed sampling domain, each time a new solution is found.

Thus, we see that the common challenge of RRTs related to a slow convergence rate towards the optimum, is increasing when applying the algorithms to large and complex environments. This is partially due to the sampling inefficiency and node rejection rate issue found in most variants, and for which a significant amount of solutions have been proposed with varying levels of success [24]. Also, for real-time systems and applications where time is a resource, the incremental tree wiring and re-wiring induce a computational cost that must be weighted against performance and optimality. Thus, we deem computationally constrained RRT-based planners without sophisticated sampling strategies and without coupling to graph-search-based methods such as A*, to be more suitable for smaller problems, or problems where obstacles occupy a smaller portion of the configuration space, or where non-optimal solutions are acceptable. For planning in higher

dimensional space, it is also necessary to consider other metrics than the Euclidean one [14].

Based on these issues, some main pointers towards making stand-alone RRT algorithms for ship trajectory planning scalable to larger problem sizes are given in the following.

- Simplifying polygons from the ENC data used in the planners even further for large maps where details on all curbs and bends are seldom necessary, using e.g. the Ramer-Douglas-Peucker algorithm [43]. This will effectively reduce run-time in collision checking routines.
- Make the model sampling time and maximum steering time used in the RRT steering routines adaptive based on the volume of grounding hazards present within a domain including the starting state and a new sampled state. This can cut run-time substantially by e.g. allowing long forward simulations with coarse time steps in open waters towards new states that can be easily steered towards.
- For IRRT*, incrementally prune the safe sea CDT triangles inside the informed sampling heuristic as the domain shrinks in volume and better solutions are found, and use these for sampling new safe sea states after the first initial solution is found. This can reduce the sampling rejection rate of vanilla IRRT* substantially.

5.4.2 Tuning Considerations

The parameter selection process for the algorithms were mostly based on trial and error. However, for PQ-RRT*, some extra concerns had to be taken into account which was not a large issue in the original paper [17]. For the PQ-RRT* with nonholonomic steering, we saw a large increase in computational effort due to the ancestry consideration. This was viable for the smaller planning case, but proved to be more limiting in the larger case unless the nonholonomic steering functionality is disabled or higher simulation time steps are used. Also, the algorithm run-time and performance are highly dependent on the sample adjustment procedure, itself being dependent on the three parameters N_{sa}^{max} , λ_{sa} , and d_{margin} . In total, this leads to a lower tree node number and can in the worst cases prevent the algorithm from finding solutions. Choosing λ_{sa} too small yields negligible gain from the APF-based adjustments, and requires a higher iteration number N_{sa}^{max} , which again gives higher algorithm run-time due to an increased number of obstacle distance calculations being required. Conversely, a large λ_{sa} can lead to rapid convergence towards local minima. We found that a selection of λ_{sa} in the order of 0.1% of the map width, and an adjustment number of around $N_{sa}^{max} \approx 50$ gave a reasonable trade-off between run-time and solution quality for the larger planning case in this work. For the clearance parameter d_{margin} it was found necessary to select a smaller value of around 0.5 m, as

a high value can prevent the planner from wiring into more confined areas. Because of these factors, we found the tuning of PQ-RRT* to be significantly more challenging than the other variants. Judging from the marginal improvements found in this work with consideration of nonholonomic steering, we argue that it was easier to employ IRRT* or RRT*, which required less tuning effort and achieved comparable performance. In general, we note that the parameter selection is dependent on the vehicle steering system.

A common tuning challenge was the selection of the nearest neighbour search parameter γ , also mentioned in [14], which has a large influence on performance. As the ball volume reduces with the tree size, too small values of γ can lead to negligible search radius and can in the worst case effectively reduce optimal RRT variants to the baseline RRT, where no nearest neighbors other than the closest one are considered in the wiring. A solution to consider is to bound the search radius from below, or for simplicity consider a fixed search radius.

6 Conclusion

In this article, multiple algorithms for ship trajectory planning based on RRT have been developed and compared with respect to trajectory length and computational time. The comparison focuses on varying degrees of difficulty in a complex environment containing many non-convex grounding hazards, as opposed to the often simple environments used for testing in previous work.

Practical aspects to consider when employing such algorithms in the maritime domain are also outlined and discussed, to the benefit of researchers and practitioners in the field. It is also shown through an example case that RRT variants can be beneficial in the context of automatic test scenario generation, where target ship trajectories can be sampled efficiently directly from the nodes of a built RRT. The tree can alternatively be used to sample intention scenarios for use in intelligent CAS.

From Monte Carlo simulations on selected cases, we see that PQ-RRT* attains more distance optimal trajectories, also verified through pair-wise hypothesis testing with Welch's t -test when using a significance level $\alpha = 0.05$. Here, IRRT* and RRT* follow close behind. This distance optimality comes naturally at the cost of increased run-time due to nearest neighbor searches and parent consideration in both tree wiring and rewiring. IRRT* struggles with cases where obstacles cover a large part of the configuration space. In larger planning cases, more efficient sampling procedures are needed for optimal RRT* variants to be viable due to the significant space of configurations that must be covered, causing a similar curse of dimensionality issue. This causes inefficiencies in the trade-off between computational effort,

available memory, and performance, as the optimal variants will then spend the majority of their effort wiring and rewiring their tree.

From the results and through tuning of the algorithm, it was found that the PQ-RRT* involves much higher complexity in tuning than the other variants, because of the sample adjustment procedure and ancestor consideration. On the other hand, the IRRT* algorithm here attains a good balance between simpler tuning and obtainable performance. It's informed sampling heuristic should, however, be improved to reduce its sample rejection rate. In the maritime domain, this can be achieved by an iterative pruning or update of a safe sea triangulation used to sample new collision-free configurations.

Author Contributions Algorithm development and testing, and manuscript writing and preparation were done by Trym Tengesdal. Tom Arne Pedersen and Tor Arne Johansen provided feedback and supervision on the methodology, results, and first manuscript drafts.

Funding Open access funding provided by NTNU Norwegian University of Science and Technology (incl St. Olavs Hospital - Trondheim University Hospital). The work was supported by the Research Council of Norway through the Autoship SFI, project number 309230, and by Kongsberg Maritime through the University Technology Center on Ship Performance and Cyber Physical Systems.

Data Availability The authors cannot share code repositories for the software producing the results in this work due to Intellectual Property Rights (IPR) agreements within the Autoship SFI, and agreements with Kongsberg Maritime.

Declarations

Competing Interests The authors have no relevant financial or non-financial interests to disclose.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Tengesdal, T., Rothmund, S.V., Basso, E.A., Johansen, T.A., Schmidt-Didlaukies, H.: Obstacle intention awareness in automatic collision avoidance: Full scale experiments in confined waters. *Field Robot.* **4**(1), 211–245 (2024). <https://doi.org/10.55417/fr.2024007>

2. Wang, X., Liu, J., Peng, H., Qie, X., Zhao, X., Lu, C.: A simultaneous planning and control method integrating apf and mpc to solve autonomous navigation for usvs in unknown environments. *J. Intell. Robot. Syst.* **105**(2), 36 (2022)
3. Wang, X., Deng, Z., Peng, H., Wang, L., Wang, Y., Tao, L., Lu, C., Peng, Z.: Autonomous docking trajectory optimization for unmanned surface vehicle: A hierarchical method. *Ocean Eng.* **279**, 114156 (2023)
4. IMO, C.: Convention on the International Regulations for Preventing Collisions At SEA, 1972. (2003)
5. Pedersen, T.A., Glomsrud, J.A., Ruud, E.-L., Simonsen, A., Sandri, J., Eriksen, B.-O.H.: Towards simulation-based verification of autonomous navigation systems. *Saf. Sci.* **129**, 104799 (2020)
6. Hart, P.E., Nilsson, N.J., Raphael, B.: A formal basis for the heuristic determination of minimum cost paths. *IEEE Trans. Syst. Sci. Cybern.* **4**(2), 100–107 (1968)
7. Karaman, S., Frazzoli, E.: Optimal kinodynamic motion planning using incremental sampling-based methods. In: 49th IEEE Conference on Decision and Control (CDC), pp. 7681–7687 (2010). <https://doi.org/10.1109/CDC.2010.5717430>
8. Lindemann, S.R., LaValle, S.M.: Incrementally reducing dispersion by increasing voronoi bias in rrt*. In: IEEE International Conference on Robotics and Automation, 2004. Proceedings. ICRA '04. 2004, vol. 4, pp. 3251–32574 (2004). <https://doi.org/10.1109/ROBOT.2004.1308755>
9. Vagale, A., Bye, R.T., Ouchekh, R., Osen, O.L., Fossen, T.I.: Path planning and collision avoidance for autonomous surface vehicles ii: A comparative study of algorithms. *J. Mar. Sci. Technol.* (2021). <https://doi.org/10.1007/s00773-020-00790-x>
10. Huang, Y., Chen, L., Chen, P., Negenborn, R.R., van Gelder, P.H.A.J.M.: Ship collision avoidance methods: State-of-the-art. *Saf. Sci.* **121**, 451–473 (2020)
11. Noreen, I., Khan, A., Habib, Z.: Optimal path planning using rrt* based approaches: a survey and future directions. *Int. J. Adv. Comput. Sci. Appl.* **7**(11) (2016)
12. Kavraki, L.E., Svestka, P., Latombe, J.-C., Overmars, M.H.: Probabilistic roadmaps for path planning in high-dimensional configuration spaces. *IEEE Trans. Robot. Autom.* **12**(4), 566–580 (1996)
13. LaValle, S.M., et al.: Rapidly-exploring random trees: A new tool for path planning (1998)
14. Noreen, I., Khan, A., Habib, Z.: A comparison of rrt, rrt* and rrt*-smart path planning algorithms. *Int. J. Comput. Sci. Netw. Secur. (IJCSNS)* **16**(10), 20 (2016)
15. Nasir, J., Islam, F., Malik, U., Ayaz, Y., Hasan, O., Khan, M., Muhammad, M.S.: Rrt*-smart: A rapid convergence implementation of rrt*. *Int. J. Adv. Robot. Syst.* **10**(7), 299 (2013). <https://doi.org/10.5772/56718>
16. Gammell, J.D., Srinivasa, S.S., Barfoot, T.D.: Informed rrt: Optimal sampling-based path planning focused via direct sampling of an admissible ellipsoidal heuristic. In: 2014 IEEE/RSJ International Conference on Intelligent Robots and Systems, pp. 2997–3004 (2014). IEEE
17. Li, Y., Wei, W., Gao, Y., Wang, D., Fan, Z.: Pq-rrt*: An improved path planning algorithm for mobile robots. *Expert Syst. Appl.* **152**, 113425 (2020)
18. Qureshi, A.H., Ayaz, Y.: Potential functions based sampling heuristic for optimal path planning. *Auton. Robot.* **40**, 1079–1093 (2016)
19. Jeong, I.-B., Lee, S.-J., Kim, J.-H.: Quick-rrt*: Triangular inequality-based implementation of rrt* with improved initial solution and convergence rate. *Expert Syst. Appl.* **123**, 82–90 (2019)
20. Klemm, S., Oberländer, J., Hermann, A., Roennau, A., Schamm, T., Zollner, J.M., Dillmann, R.: Rrt*-connect: Faster, asymptotically optimal motion planning. In: 2015 IEEE International Conference on Robotics and Biomimetics (ROBIO), pp. 1670–1677 (2015). IEEE
21. Chiang, H.-T.L., Hsu, J., Fiser, M., Tapia, L., Faust, A.: Rl-rrt: Kinodynamic motion planning via learning reachability estimators from rl policies. *IEEE Robot. Autom. Lett.* **4**(4), 4298–4305 (2019). <https://doi.org/10.1109/LRA.2019.2931199>
22. Wang, J., Chi, W., Li, C., Wang, C., Meng, M.Q.-H.: Neural rrt*: Learning-based optimal path planning. *IEEE Trans. Autom. Sci. Eng.* **17**(4), 1748–1758 (2020). <https://doi.org/10.1109/TASE.2020.2976560>
23. Lai, T., Ramos, F., Francis, G.: Balancing global exploration and local-connectivity exploitation with rapidly-exploring random disjointed-trees. In: 2019 International Conference on Robotics and Automation (ICRA), pp. 5537–5543 (2019). <https://doi.org/10.1109/ICRA.2019.8793618>
24. Vêras, L.G.D., Medeiros, F.L., Guimarães, L.N.: Systematic literature review of sampling process in rapidly-exploring random trees. *IEEE Access* **7**, 50933–50953 (2019)
25. Dang, T., Donze, A., Maler, O., Shalev, N.: Sensitive state-space exploration. In: 2008 47th IEEE Conference on Decision and Control, pp. 4049–4054 (2008). <https://doi.org/10.1109/CDC.2008.4739371>
26. Tuncali, C.E., Fainekos, G.: Rapidly-exploring random trees for testing automated vehicles. In: 2019 IEEE Intelligent Transportation Systems Conference (ITSC), pp. 661–666 (2019). <https://doi.org/10.1109/ITSC.2019.8917375>
27. Koschi, M., Pek, C., Maierhofer, S., Althoff, M.: Computationally efficient safety falsification of adaptive cruise control systems. In: 2019 IEEE Intelligent Transportation Systems Conference (ITSC), pp. 2879–2886 (2019). <https://doi.org/10.1109/ITSC.2019.8917287>
28. Minne, P.K.E.: Automatic testing of maritime collision avoidance algorithms. Master's thesis, NTNU (2017)
29. Pedersen, T.A., Neverlien, Å., Glomsrud, J.A., Ibrahim, I., Mo, S.M., Rindarøy, M., Torben, T., Rokseth, B.: Evolution of safety in marine systems: From system-theoretic process analysis to automated test scenario generation. *J. Phys.: Conf. Ser.* **2311**(1), 012016 (2022). <https://doi.org/10.1088/1742-6596/2311/1/012016>
30. Torben, T.R., Glomsrud, J.A., Pedersen, T.A., Utne, I.B., Spørensen, A.J.: Automatic simulation-based testing of autonomous ships using gaussian processes and temporal logic. *Proc. Inst. Mech. Eng. Part O: J. Risk Reliab.* **0**(0) (2022). <https://doi.org/10.1177/1748006X211069277>
31. Bolbot, V., Gkerekos, C., Theotokatos, G., Boulougouris, E.: Automatic traffic scenarios generation for autonomous ships collision avoidance system testing. *Ocean Eng.* **254**, 111309 (2022)
32. Zhu, F., Zhou, Z., Lu, H.: Randomly testing an autonomous collision avoidance system with real-world ship encounter scenario from ais data. *J. Mar. Sci. Eng.* **10**(11) (2022). <https://doi.org/10.3390/jmse10111588>
33. Porres, I., Azimi, S., Lilius, J.: Scenario-based testing of a ship collision avoidance system. In: 2020 46th Euromicro Conference on Software Engineering and Advanced Applications (SEAA), pp. 545–552 (2020). <https://doi.org/10.1109/SEAA51224.2020.00090>
34. Tengesdal, T., Johansen, T.A., Grande, T.D., Blindheim, S.: Ship collision avoidance and anti grounding using parallelized cost evaluation in probabilistic scenario-based model predictive control. *IEEE Access* **10**, 111650–111664 (2022). <https://doi.org/10.1109/ACCESS.2022.3215654>
35. Breivik, M., Fossen, T.I.: Guidance laws for planar motion control. In: Proc. 47th IEEE Conf. Decision and Control, pp. 570–577 (2008). <https://doi.org/10.1109/CDC.2008.4739465>
36. Karaman, S., Walter, M.R., Perez, A., Frazzoli, E., Teller, S.: Any-time motion planning using the rrt*. In: 2011 IEEE International Conference on Robotics and Automation, pp. 1478–1483 (2011). <https://doi.org/10.1109/ICRA.2011.5980479>

37. Chiang, H.L., Tapia, L.: COLREG-RRT: An RRT-based COLREGS-compliant motion planner for surface vehicle navigation. *IEEE Robot. Autom. Lett.* **3**(3), 2024–2031 (2018). <https://doi.org/10.1109/LRA.2018.2801881>
38. Enevoldsen, T.T., Reinartz, C., Galeazzi, R.: Colregs-informed rrt* for collision avoidance of marine crafts. In: 2021 IEEE International Conference on Robotics and Automation (ICRA), pp. 8083–8089 (2021). IEEE
39. Enevoldsen, T.T., Blanke, M., Galeazzi, R.: Sampling-based collision and grounding avoidance for marine crafts. *Ocean Eng.* **261**, 112078 (2022)
40. Guttman, A.: R-trees: A dynamic index structure for spatial searching. In: Proceedings of the 1984 ACM SIGMOD International Conference on Management of Data, pp. 47–57 (1984)
41. Beckmann, N., Kriegel, H.-P., Schneider, R., Seeger, B.: The r*-tree: An efficient and robust access method for points and rectangles. In: Proceedings of the 1990 ACM SIGMOD International Conference on Management of Data, pp. 322–331 (1990)
42. Bentley, J.L.: Multidimensional binary search trees used for associative searching. *Commun. ACM* **18**(9), 509–517 (1975)
43. Ramer, U.: An iterative procedure for the polygonal approximation of plane curves. *Comput. Graph. Image Process.* **1**(3), 244–256 (1972)
44. Blindheim, S., Johansen, T.A.: Electronic navigational charts for visualization, simulation, and autonomous ship control. *IEEE Access* **10**, 3716–3737 (2022). <https://doi.org/10.1109/ACCESS.2021.3139767>
45. Zhang, Z., Zhang, Y., Han, R., Zhang, L., Pan, J.: A generalized continuous collision detection framework of polynomial trajectory for mobile robots in cluttered environments. *IEEE Robot. Autom. Lett.* **7**(4), 9810–9817 (2022). <https://doi.org/10.1109/LRA.2022.3191934>
46. Yang, K., Moon, S., Yoo, S., Kang, J., Doh, N.L., Kim, H.B., Joo, S.: Spline-based rrt path planner for non-holonomic robots. *J. Intell. Robot. Syst.* **73**(1–4), 763–782 (2014)
47. Perez, A., Platt, R., Konidaris, G., Kaelbling, L., Lozano-Perez, T.: Lqr-rrt*: Optimal sampling-based motion planning with automatically derived extension heuristics. In: 2012 IEEE International Conference on Robotics and Automation, pp. 2537–2542 (2012). IEEE
48. Yavari, M., Gupta, K., Mehrandezh, M.: Lazy steering rrt*: An optimal constrained kinodynamic neural network based planner with no in-exploration steering. In: 2019 19th International Conference on Advanced Robotics (ICAR), pp. 400–407 (2019). <https://doi.org/10.1109/ICAR46387.2019.8981551>
49. Tengesdal, T., Johansen, T.A.: Simulation framework and software environment for evaluating automatic ship collision avoidance algorithms*. In: 2023 IEEE Conference on Control Technology and Applications (CCTA), pp. 186–193 (2023). <https://doi.org/10.1109/CCTA54093.2023.10252863>
50. Ruxton, G.D.: The unequal variance t-test is an underused alternative to student's t-test and the Mann-Whitney u test. *Behav. Ecol.* **17**(4), 688–690 (2006)
51. Chauvin, C., Lardjane, S.: Decision making and strategies in an interaction situation: Collision avoidance at sea. *Transp. Res. Part F: Traffic Psychol. Behav.* **11**(4), 259–269 (2008)
52. Kuwata, Y., Wolf, M.T., Zarzhitsky, D., Huntsberger, T.L.: Safe maritime autonomous navigation with COLREGS, using velocity obstacles. *IEEE J. Ocean. Eng.* **39**(1), 110–119 (2014). <https://doi.org/10.1109/JOE.2013.2254214>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Trym Tengesdal received the M.Sc. and Ph.D. degrees in engineering cybernetics from the Norwegian University of Science and Technology (NTNU), Trondheim, Norway, in 2019 and 2022, respectively. During the period from 2022 to 2024 he was a Postdoctoral Researcher at NTNU, affiliated with the Autoship Centre for Research-Based Innovation (SFI), working on topics within ship motion planning, machine learning, and data-driven automatic collision avoidance. Currently, he is an R&D engineer at AutoStore.

Tom Arne Pedersen has a M.Sc. (2002) and Ph.D. (2009) in Marine Technology from the Norwegian University of Science and Technology (NTNU). He has worked for Marine Cybernetics since 2008, holding positions as senior project engineer, R&D Manager Drilling Systems and Product Manager Drilling Systems. Marine Cybernetics was acquired by DNV in 2014 and Tom Arne Pedersen currently holds the position as Principal Researcher in DNV Group Technology and Research, working on a testing framework for autonomous navigation systems.

Tor Arne Johansen received the M.Sc. degree in 1989 and the Ph.D. degree in 1994, both in electrical and computer engineering, from the Norwegian University of Science and Technology, Trondheim, Norway. From 1995 to 1997, he worked at SINTEF as a researcher before he was appointed Associated Professor at the Norwegian University of Science and Technology in Trondheim in 1997 and Professor in 2001. He has published several hundred articles in the areas of control, estimation and optimization with applications in the marine, aerospace, automotive, biomedical and process industries. In 2002 Johansen co-founded the company Marine Cybernetics AS where he was Vice President until 2008. Prof. Johansen received the 2006 Arch T. Colwell Merit Award of the SAE, and is currently a principal researcher within the Center of Excellence on Autonomous Marine Operations and Systems (NTNU-AMOS) and director of the Unmanned Aerial Vehicle Laboratory at NTNU and the SmallSat Laboratory at NTNU. He recently co-founded the spin-off companies Scout Drone Inspection, UBIQ Aerospace, Zeabuz, and SentiSystems.